

Deep Learning (CAI-466)

Objectives:

- Object detection and localization
 - Convolutional Implementation of Sliding Windows
-

Object detection and Localization

One of the toughest but hottest field of computer vision is Object detection. Let's learn how we can apply our knowledge of CNNs in it.

Object Localization

Object detection is one of the areas in which deep learning is doing great from last few years. Let's start with understanding what are localization and detection?

- **Image Classification:**

Classify an image to a specific class. The whole image represents one class. We don't want to know exactly where is the object. Usually only one object is presented.



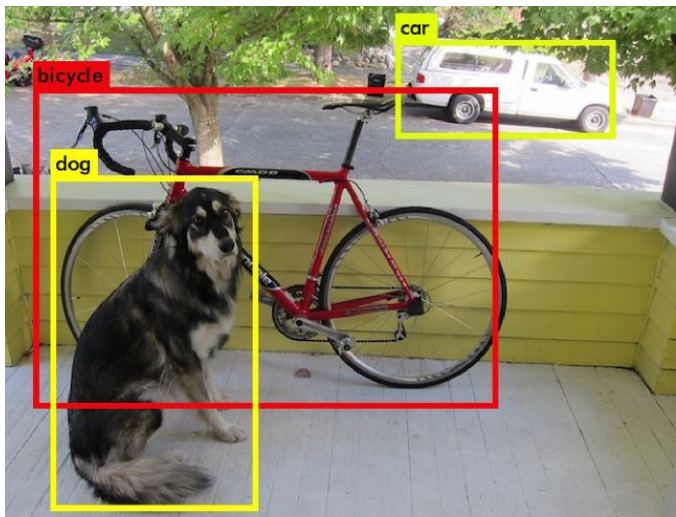
- **Classification with localization:**

Given an image we want to learn the class of the image and where are the class location in the image. We need to detect a class and a rectangle of where that object is. Usually only one object is presented.



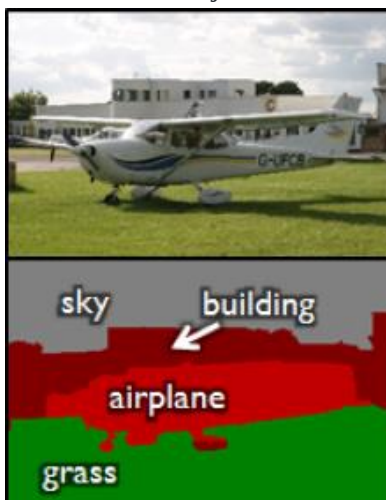
- **Object detection:**

Given an image we want to detect all the objects in the image that belong to a specific class and give their location. An image can contain more than one objects with different classes.



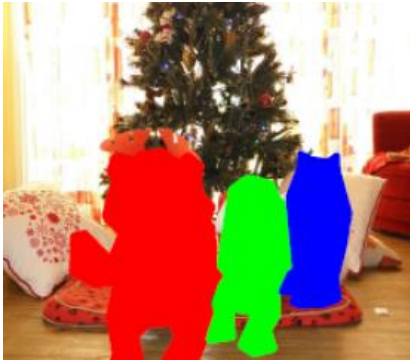
- **Semantic Segmentation:**

We want to Label each pixel in the image with a category label. Semantic Segmentation don't differentiate instances, only care about pixels. It detects no objects just pixels. If there are two objects of the same class is intersected, we won't be able to separate them.



- **Instance Segmentation**

This is like the full problem. Rather than we want to predict the bounding box, we want to know which pixel label but also distinguish them.



- **Landmark Detection**

In some of the computer vision problems you will need to output some points. That is called landmark detection. For example, if you are working in a face recognition problem you might want some points on the face like corners of the eyes, corners of the mouth, and corners of the nose and so on. This can help in a lot of application like detecting the pose of the face.

Y shape for the face recognition problem that needs to output 64 landmarks is given below:

```
Y = [
    ThereIsAface                # Probability of face is presented 0 or 1
    l1x,
    l1y,
    .....,
    l64x,
    l64y
]
```

In your labeled data, if l1x,l1y is the left corner of left eye, all other l1x,l1y of the other examples has to be the same.

Object Detection

To make image classification we use a Conv Net with Softmax attached to the end of it.

To make classification with localization we use a Conv Net with Softmax attached to the end of it and a four numbers **bx** , **by** , **bh** , and **bw** to tell you the location of the class in the image.

The dataset should contain these four numbers with the class too. Defining the target label Y in classification with localization problem:

```

Y = [
    Pc          # Probability of an object is presented
    bx          # Bounding box
    by          # Bounding box
    bh          # Bounding box
    bw          # Bounding box
    c1          # The classes
    c2
    ...
]

```

Example: (Object is present)

```

Y = [
    1          # Object is present
    0
    0
    100
    100
    0
    1
    0
]

```

Example: (Object is not present)

```

Y = [
    0          # Object isn't presented
    ?          # ? means we dont care with other values
]

```

The loss function for the Y we have created (Example of the square error):

```

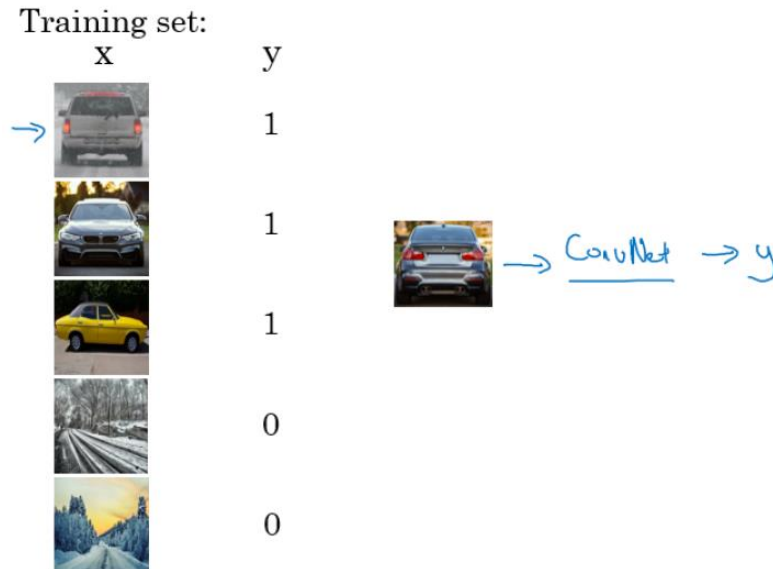
L(y',y) = {
    (y1'-y1)^2 + (y2'-y2)^2 + ...      if y1 = 1
    (y1'-y1)^2                        if y1 = 0
}

```

In practice we use logistic regression for **pc** , log likely hood loss for classes, and squared error for the bounding box.

We will use a Conv net to solve the object detection problem using a technique called the sliding windows detection algorithm. For example let's say we are working on Car object detection.

The first thing, we will train a Conv net on cropped car images and non car images.



After we finish training of this Conv net we will then use it with the sliding windows technique.

Sliding windows detection algorithm:

- i. Decide a rectangle size.
- ii. Split your image into rectangles of the size you picked. Each region should be covered. You can use some strides.
- iii. For each rectangle feed the image into the Conv net and decide if It is a car or not.
- iv. Pick larger/smaller rectangles and repeat the process from 2 to 3.
- v. Store the rectangles that contain the cars.
- vi. If two or more rectangles intersect choose the rectangle with the best accuracy.

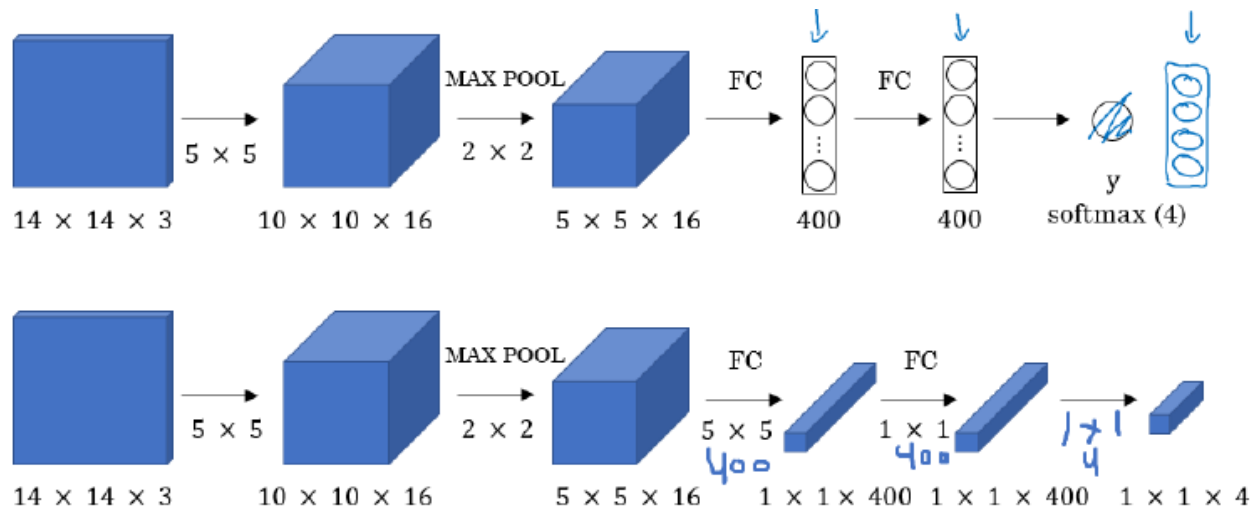
Disadvantage of sliding window is the computation time.

In the era of machine learning before deep learning, people used a hand crafted linear classifiers that classify the object and then use the sliding window technique. The linear classifier make it a cheap computation. But in the deep learning era that is so computational expensive due to the complexity of the deep learning model.

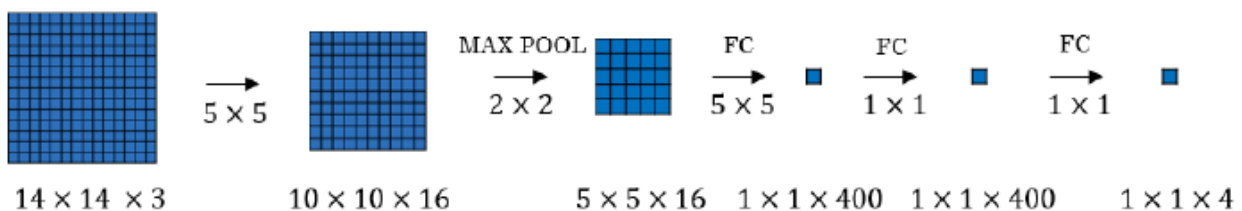
To solve this problem, we can implement the sliding windows with a *Convolutional approach*. One other idea is to compress your deep learning model.

Convolutional Implementation of Sliding Windows

Turning FC layer into convolutional layers (predict image class from four classes):

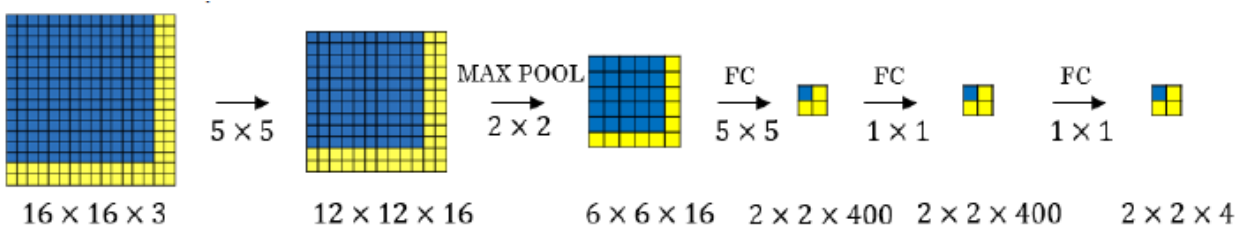


As you can see in the above image, we turned the FC layer into a Conv layer using a convolution with the width and height of the filter is the same as the width and height of the input. First let's consider that the Conv net you trained is like this (No FC all is conv layers):



Example:

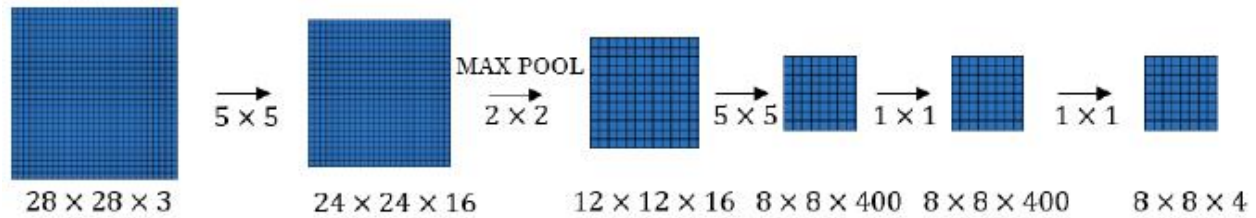
Say now we have a $16 \times 16 \times 3$ image that we need to apply the sliding windows in. By the normal implementation that has been mentioned previously, we would run this Conv net four times each rectangle size will be 16×16 . The convolution implementation will be as follows:



Simply we have feed the image into the same Conv net we have trained.

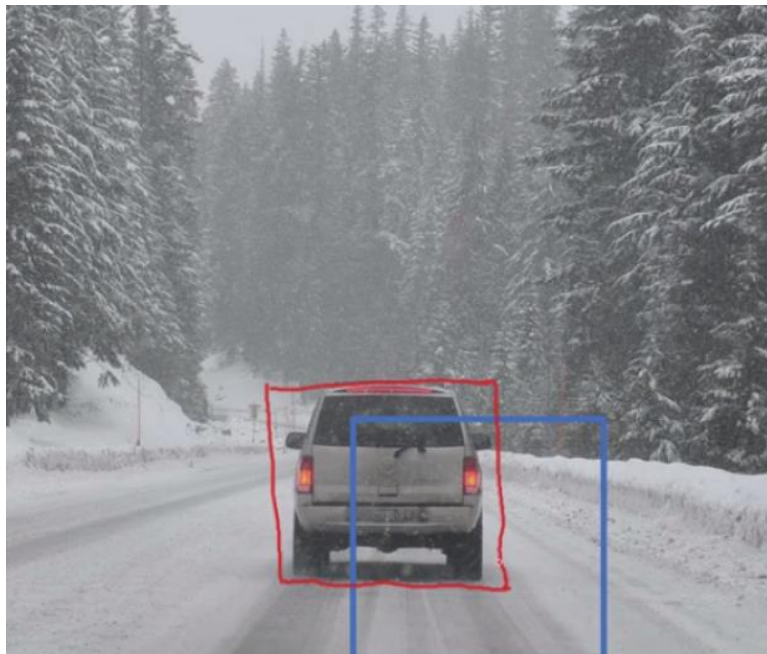
The left cell of the result "The blue one" will represent the first sliding window of the normal implementation. The other cells will represent the others. Its more efficient because it now shares the computations of the four times needed.

Another example would be:



This example has a total of 16 sliding windows that shares the computation together.

The weakness of the algorithm is that the position of the rectangle would not be so accurate. May be none of the rectangles is exactly on the object you want to recognize. In red, the rectangle we want and in blue is the required car rectangle.

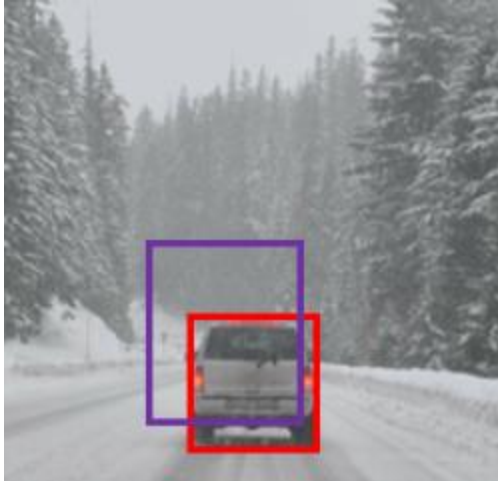


Intersection Over Union

Intersection Over Union is a function used to evaluate the object detection algorithm.

It computes size of intersection and divide it by the union. More generally, *IoU* is a measure of the overlap between two bounding boxes.

For example:



The red is the labeled output and the purple is the predicted output.

To compute Intersection Over Union we first compute the union area of the two rectangles which is "the first rectangle + second rectangle" Then compute the intersection area between these two rectangles.

Finally $IOU = \text{intersection area} / \text{Union area}$

If $IOU \geq 0.5$ then its good. The best answer will be 1.

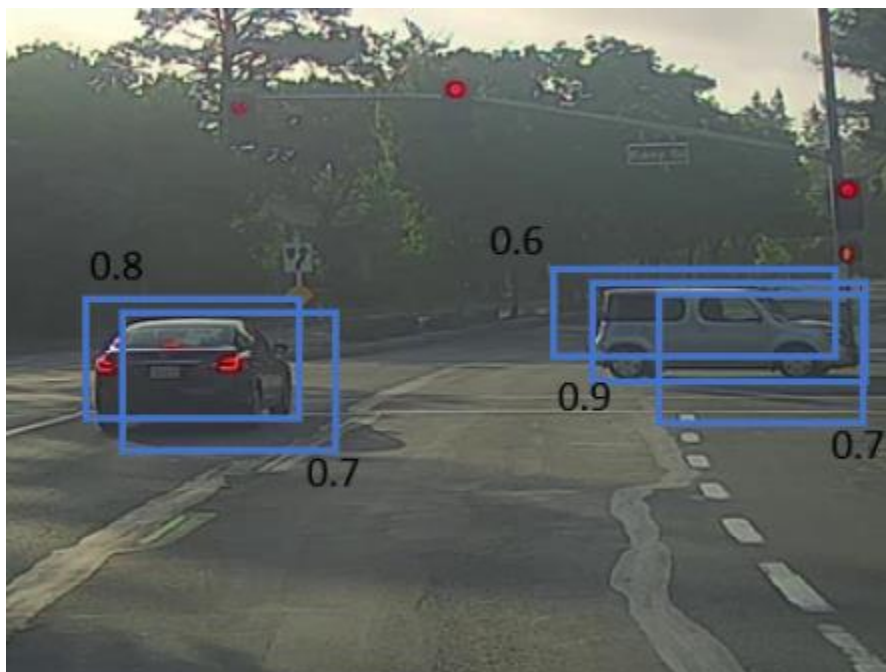
The higher the IOU the better is the accuracy.

Non-max Suppression

One of the problems we have addressed in YOLO is that it can detect an object multiple times.

Non-max Suppression is a way to make sure that YOLO detects the object just once.

For example:



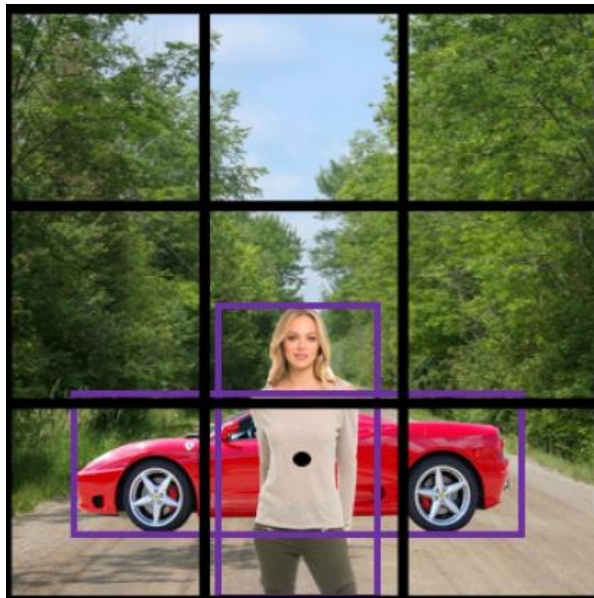
Each car has two or more detections with different probabilities. This came from some of the grids that thinks that this is the center point of the object.

Non-max suppression algorithm:

- i. Lets assume that we are targeting one class as an output class.
 - ii. Y shape should be $[P_c, b_x, b_y, b_h, b_w]$ Where P_c is the probability if that object occurs.
 - iii. Discard all boxes with $P_c < 0.6$
 - iv. While there are any remaining boxes:
 - a. Pick the box with the largest P_c Output that as a prediction.
 - b. Discard any remaining box with $IoU > 0.5$ with that box output in the previous step i.e any box with high overlap(greater than overlap threshold of 0.5).
- If there are multiple classes/object types c you want to detect, you should run the Non-max suppression c times, once for every output class.

Anchor Boxes

In YOLO, a grid only detects one object. What if a grid cell wants to detect multiple object?



Car and person grid is same here.

In practice this happens rarely.

The idea of Anchor boxes helps us solving this issue.

If $Y = [P_c, b_x, b_y, b_h, b_w, c_1, c_2, c_3]$ Then to use two anchor boxes like this:

$Y = [P_c, b_x, b_y, b_h, b_w, c_1, c_2, c_3, P_c, b_x, b_y, b_h, b_w, c_1, c_2, c_3]$ We simply have repeated the one anchor Y .

The two anchor boxes you choose should be known as a shape:

Anchor box 1: Anchor box 2:



So Previously, each object in training image is assigned to grid cell that contains that object's midpoint.

With two anchor boxes, Each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU. You have to check where your object should be based on its rectangle closest to which anchor box.

Example of data:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Handwritten annotations for the second vector (orange):

- b_x (orange)
- b_y (orange)
- b_h (orange)
- b_w (orange)
- c_1 (orange)
- c_2 (orange)
- c_3 (orange)

Handwritten annotations for the third vector (green):

- b_x (green)
- b_y (green)
- b_h (green)
- b_w (green)
- c_1 (green)
- c_2 (green)
- c_3 (green)

Handwritten annotations for the fourth vector (blue):

- c_1 (blue)
- c_2 (blue)
- c_3 (blue)

Handwritten notes:

- car only?
- anchor box 1 (bracketed next to the first vector)
- anchor box 2 (bracketed next to the second vector)

Where the car was near the anchor 2 than anchor 1.

You may have two or more anchor boxes but you should know their shapes.

how do you choose the anchor boxes and people used to just choose them by hand. Maybe five or ten anchor box shapes that spans a variety of shapes that cover the types of objects you seem to detect frequently.

You may also use a k-means algorithm on your dataset to specify that.

Anchor boxes allows your algorithm to specialize, means in our case to easily detect wider images or taller ones.